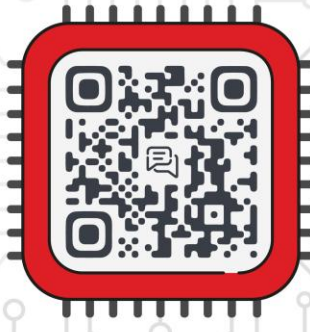




نظري

6



14



2100 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Advanced Pipelining

د. سيرا أستور

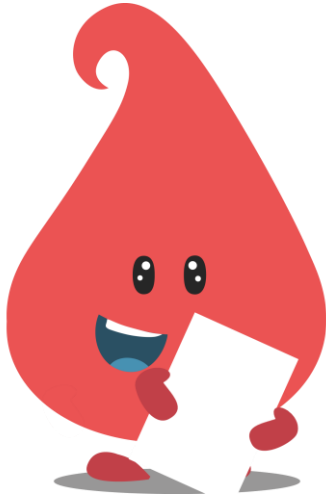


RB Informatics; 17/11/2025

بنیان الحواسيب 2

الأفكار الرئيسية في المحاضرة :

- مراجعة لمفهوم الـ Pipeline
- أنواع الـ Hazards
 - ↳ Data Hazards + أنواعها (RAW, WAW, WAR, RAR)
 - ↳ Control Hazards
 - ↳ Structural Hazards
- طرق حل المشاكل مثل register renaming
- Exceptions & Program behavior
- Multiple Issue Processors
- Static/Dynamic Issue و VLIW
- Program correctness



لنبدأ معا بمراجعة بسيطة للأفكار السابقة :

المعالج يعمل بطريقة الـ Pipeline اي يقسم تنفيذ التعليمات إلى مراحل وينفذ عدة تعليمات معا كل واحدة في مرحلة مختلفة لذلك إذا كانت تعليمة تحتاج قيمة لم يتم حسابها بعد لأن التعليمة السابقة ما زالت في الـ Pipeline أو لأسباب أخرى يحصل Hazard

أنواع الـ Hazard :

1. Structural Hazards:

إذا كان أحد أجزاء الهاردوير مطلوب من أكثر من تعليمة بنفس الوقت.

مثلا: لدينا وحدة Memory واحدة والتعليمة الحالية تحتاج جلب من Memory (IF) وتعليمة ثانية تحتاج قراءة بيانات من Memory (MEM) سيحصل لدينا Hazard

والحل مضاعفة الهاردوير أي عمل separate instruction & data memory

ملاحظة :

معالجات MIPS تستخدم ذاكرتين منفصلتين: ذاكرة للتعليمات/ وذاكرة للبيانات

لذلك عندما ينفذ الـ Pipeline تعليمات:

- مرحلة IF تحتاج قراءة من Instruction Memory
- مرحلة MEM تحتاج قراءة/كتابة من Data Memory

بما أنهما منفصلتان فلن يحدث تضارب على الذاكرة بين المرحلتين، وبالتالي لا يوجد Structural Hazard من هذا النوع.

2. Control Hazards:

- تنفيذ تعليمة معينة يعتمد على قرار الفرع قبله ويحصل عند تعليمات BEQ, BNE, J, JR لأن المعالج لا يعلم هل يذهب للمسار الجديد branch taken أم يكمل بشكل طبيعي not taken والحل هنا
- Branch Prediction (توقع) / Flushing (إلغاء تعليمات)

التعليمات :

1. تعليمة معتمدة على فرع:

لا يمكن نقلها قبل الفرع، لأن تنفيذها لن يكون تحت سيطرة الفرع.

2. تعليمة غير معتمدة على فرع:

لا يمكن نقلها بعد الفرع، لأن تنفيذها سيصبح معتمداً على الفرع (وهو خطأ).

مثال:



```
if (x == 0) {
    هذه التعليمة معتمدة على الفرع
    y = 1;
}
هذه غير معتمدة على الفرع
z = 2;

if y = 1 لا يمكن نقلها قبل
z = 2 لا يمكن نقلها داخل if (إلا إذا كان من
الممكن تنفيذها دائماً)
```

3. Data Hazards:

يحصل عندما تحتاج تعليمة نتيجة تعليمة سابقة لم تصل بعد

تُصنف أنواع الـ data hazards في المعالجات إلى ثلاثة أنواع رئيسية بناءً على كيفية تأثيرها على تدفق البيانات بين التعليمات هذه الأنواع هي:

- True Data Dependence (RAW = Read After Write)

يحدث عندما تعتمد تعليمة على نتيجة تعليمة سابقة أي أن التعليمة اللاحقة تحتاج إلى قراءة قيمة تم كتابتها بواسطة تعليمة سابقة

مثال:

يكتب في R1 ADD R1, R2, R3 # i
 SUB R4, R1, R5 # j يقرأ من R1 قبل أن يكتمل كتابة i
 التعليمة j تحاول قراءة مصدر قبل أن تكتبه التعليمة i وسببها
 الاعتمادية الحقيقية للبيانات (True Data Dependence).

وهي تعتمد على إعادة استخدام اسم السجل فقط وهي ليست اعتمادية حقيقية (True Data Dependence) لأنها لا تمثل اعتماداً على البيانات نفسها بل على الاسم فقط وتنقسم إلى:

1. Anti-dependence (WAR (Write After Read

- التعليمة j تحاول الكتابة في ناتج قبل أن تقرأ التعليمة i
- النتيجة: التعليمة i تحصل على القيمة الجديدة (الخطئة) بدلاً من القديمة
- سببها: الاعتمادية العكسية (Anti-dependence)
- في معظم خطوط الأنابيب الساكنة (Static Pipelines) لا تحدث لأن:
 جميع القراءات تكون مبكرة في مرحلة فك الشفرة ID جميع الكتابات تكون متأخرة (في مرحلة الكتابة WR)

تقرأ من R1 ADD R3, R1, R2 # i
 SUB R1, R4, R5 # j تكتب في R1 قبل أن تكتمل قراءة

هنا ADD قد تحصل على قيمة R1 المعدلة من SUB بدلاً من القيمة الأصلية

2. Output dependence (WAW (Write After Write

- أي أن التعليمة j تحاول الكتابة في ناتج قبل أن تكتبه التعليمة i
- النتيجة: الكتابة تتم بترتيب خاطئ وسببها اعتمادية الإخراج (Output Dependence)

مثال:

MUL R1, R2, R3 # i تكتب في R1 (تأخذ وقتاً طويلاً)
 ADD R1, R4, R5 # j تكتب في R1 أيضاً (وتنتهي أولاً)

النتيجة: قيمة ADD تكتب أولاً ثم تستبدل بقيمة MUL وهذا يعطي نتيجة خاطئة

وفي هذا المثال يتم القراءة مرتان لنفس السجل ولكن لا تسببان أي مشكلة

مثال:

```

Loop: L.D F0,0(R1)  #
      F0 + F2 → F4  #
      ADD.D F4, F0, F2
      S.D F4, 0(R1)  #
      يكتب النتيجة في نفس المكان
      DADDUI R1, R1, #-8
      ينقص R1 بمقدار 8
      BNE R1, R2, Loop
      يقارن R1 مع R2
  
```

ملاحظة:

- لماذا True Data Dependence مهم؟ لأنه الوحيد الذي يسبب قراءة قيمة غير صحيحة إذا لم نحله.
- الاعتمادية الحقيقية (Data Dependence) لا يمكن حذفها بإعادة التسمية، لأنها تعتمد على تدفق البيانات الفعلي.
- الاعتمادية الاسمية تعتبر "زائفة" ويمكن إزالتها بتغيير الاسم فقط.
- في MIPS الـ pipeline لا يملك WAW ولا WAR لأن الكتابة دائماً تحصل في آخر مرحلة (WB) وبنفس الترتيب.

ما الحل لهذا النوع من ال hazards ؟

Register Renaming

بما أنها ليست اعتمادية حقيقية يمكن تغيير الاسم (السجل أو عنوان الذاكرة) في أحد التعليمات ويتم ذلك إما:

:Rstatically

تتم إعادة التسمية الثابتة أثناء مرحلة (compilation) بواسطة (compiler) حيث يقوم compiler بتحديد السجلات التي سيتم استخدامها لكل تعليمة مما يسمح له بتجنب أي hazards محتملة في الاعتماد (يتم تخصيص سجلات جديدة للتعليمات بناءً على تحليل compiler)

:dynamically

عن طريق hardware، حيث يتم استخدام هيكل بيانات يسمى جدول إعادة التسمية (Renaming Table) أو جدول السجلات (Register Alias Table) وعندما تنفذ تعليمة يقوم hardware بتحديد السجل الفعلي الذي يجب استخدامه بناءً على الحالة الحالية للتعليمات الأخرى.

و يمكن أن يقوم hardware بإعادة تسمية السجلات في الوقت الفعلي مما يسمح بتنفيذ التعليمات المتوازية دون التعرض للاعتمادية.

ولكن يحتاج إلى موارد إضافية مثل الذاكرة لتخزين جداول إعادة التسمية.

والحل في المثال السابق بإعادة التسمية:

...
DADDUI R4, R1, #-8 #
نستخدم R4 بدل R1 للتعديل
BNE R4, R2, Loop #
نقارن R4 بـ R1 بدل R1

كيف نحدد إن كانت تعليمة تعتمد على أخرى؟

مثلا تعليمة ز تعتمد على أ إذا:

- أ تنتج (Write) قيمة على سجل / ز تحتاج (Read) نفس السجل ويجب الحفاظ على الترتيب بينهما ز بعد أ

طرق انتقال البيانات بين التعليمات:

▪ Register Dependence:

ADD R1, R2, R3

SUB R4, R1, R5 R1 على الاعتماد

▪ Memory Dependence:

حيث عنوان الذاكرة يساوي: Base register + offset

مثال :

LW R1, 100(R4)

SW R2, 20(R6)

قد يكون:

Memory dependence R4 + 100 = نفس العنوان الذي يشير إليه R6 + 20 وفي هذه الحالة يكون هناك اعتماد

العلاقة بين الترتيب البرمجي والاعتمادية على التحكم:

الترتيب البرمجي هو ترتيب التعليمات كما هو مكتوب في البرنامج وهو مهم لأنه يؤثر على كيفية تنفيذ التعليمات وما هي القيم التي تستخدمها التعليمات اللاحقة.

الاعتمادية على التحكم تشير إلى العلاقة بين التعليمات التي تعتمد على نتائج تعليمات التحكم مثل التعليمات الشرطية (if-else) أو حلقات التكرار (loops) إذا كانت تعليمة تعتمد على نتيجة فرع (branch)، فإن تنفيذ هذه التعليمة يعتمد على ما إذا كان الفرع قد تم اتخاذه أم لا.

:Exception Behavior

المشكلة: إعادة ترتيب التعليمات يجب ألا تسبب استثناءات جديدة في البرنامج

مثال: إذا نُفذت تعليمة قبل فرع (رغم أنها تعتمد عليه) وكانت هذه التعليمة ستسبب استثناء (مثل قسمة على صفر أو خطأ في الذاكرة) فإن البرنامج سيتصرف بشكل مختلف عن السلوك الأصلي.

Data Flow:

- عند وجود تعليمات شرطية، يصبح تدفق البيانات ديناميكيًا مما يعني أن القيم التي تُستخدم في التعليمات تعتمد على نتيجة الفروع لذلك يجب أن نكون حذرين بشأن أي تعليمة سابقة يمكن أن تؤثر على القيم المستخدمة في التعليمات التالية.
- الترتيب البرمجي يُحدد أي سلف (Predecessor) سيقدم القيمة الفعلية للتعليمة لذلك إذا تم تنفيذ تعليمة تعتمد على قيمة لم تحسب بعد بسبب إعادة ترتيب التعليمات، فقد يؤدي ذلك إلى نتائج غير صحيحة.

مثال على Exception Behavior :

```

جمع                                DADDU R2, R3, R4 #
إذا R2 == 0 اذهب إلى L1            BEQZ R2, L1      #
تحميل من الذاكرة من العنوان في R2 LW R1, 0(R2)      #

```

مثال على تدفق البيانات:

```

تعيين R1                            DADDU R1, R2, R3 #
إذا R4 == 0 اذهب إلى L              BEQZ R4, L      #
DSUBU R1, R5, R6 # (فقط إذا R4 != 0)
L:
تستخدم R1                          OR R7, R1, R8    #

```

قيمة R1 التي ستصل إلى OR تعتمد على الفرع:

- إذا $R4 == 0$ فإن R1 تأتي من DADDU
- إذا $R4 != 0$ فإن R1 تأتي من DSUBU
- لا يمكن تحريك DSUBU قبل الفرع لأن ذلك سيغير تدفق البيانات دائمًا

صحة البرنامج Program Correctness:

compiler يخمن هل سيحدث فرع في البرنامج أم لا؟

- إذا كان التوقع صحيح فهذا يؤدي الى زيادة سرعة الأداء في البرنامج
- أما إذا كان التخمين خاطئ أي حصل التنفيذ بلا فائدة فهذا لا يسبب ضرر ولكن تبقى النتيجة صحيحة

التوازي بين التعليمات (pipeline):

أي تشغيل أجزاء من تعليمات مختلفة معا

ولكن كيف نزيد هذا التوازي في المعالج ؟

1. زيادة مراحل الـ pipeline

وبذلك نستطيع زيادة عدد معالجة التعليمات في نفس الوقت

2. Multiple issuing :

أي نضع في المعالج أكثر من قطعة من نفس الهاردوير مثل وجود أكثر من وحدة حسابية وبذلك تستطيع تنفيذ أكثر من تعليمة في دورة واحدة وهذا يجعل الأداء أسرع

Multiple Issue Processors:

أي إذا أصبح المعالج ينفذ عدة تعليمات معا فهذا يعني أنه يجب أن يتعامل مع 3 مشاكل :

1. Issue Slots:

أي ان المعالج يجب ان يحدد مكان كل تعليمة ولذلك بعض المعالجات تفرض ترتيب محدد للتعليمات لتسهيل العملية

2. Data and Control Hazards:

المشاكل التي تحدثنا عنها مطولا سابقا

3. Instructions Issue Policy:

المعالج قد يعمل بطريقتين:

1. In-Order Issue with In-Order Completion

أي دخول وخروج التعليمات يتم بالترتيب

2. In Order Issue with Out-of-Order Completion

أي ان الدخول بالترتيب والخروج حسب سرعة تنفيذ العمل

3. Out-of-Order issue with Out-of-Order Completion

أي ان الدخول والخروج بدون الالتزام بالترتيب حسب الأسرع وهذا يعطي أداء أعلى

Multiple Issue Processors:

وهناك نوعان:

1. Static Multiple Issue:

وهو أن الـ compiler يقرر قبل تشغيل البرنامج :

أي التعليمات يمكن تنفيذها معا

كيفية ترتيبها بحيث لا يحصل مشكلات

حيث يجمع عدة تعليمات في حزمة وتكون خلال دورة واحدة قبل بدء المعالجة وهي ما يسمى Issue Packet

*compiler يقرر كل شيء والمعالج هو الذي ينفذ

مثال VLIW:

VLIW : Very Long Instructions Word Processors

المترجم يجمع عدة تعليمات في تعليمة كبيرة والمعالج ينفذها دفعة واحدة

امثلة على معالجات VLIW:

▪ Itanium, Itanium-2

Dynamic Multiple Issue:

- أي أن القرارات تتخذ أثناء عمل البرنامج بواسطة المعالج نفسه
- أي التعليمات يمكن تنفيذها معا
- كيفية ترتيبها بحيث لا يحصل مشكلات
- كيفية إعادة ترتيبها لزيادة السرعة

مثال:

Superscalar Processors

هذه المعالجات:

- تعد خطوط التنفيذ داخلها وتعيد ترتيب التعليمات تلقائياً وتتعامل مع المشاكل (Hazards) بنفسها
- الحاسبات الحديثة (Intel, AMD, ARM) كلها Superscalar

Instruction Issue Policy:

Instruction Issue:

هي اللحظة التي تنتقل فيها التعليمة من مرحلة الـ decode إلى أول مرحلة execute أي بداية تنفيذ التعليمة داخل الوحدات الحسابية



المعالج يجب ان ينظم ترتيب:

1. جلب التعليمات من الذاكرة الى pipeline
2. تنفيذ تعليمات execute داخل وحدات التنفيذ
3. تحديث النتائج (Write-Back Order)
4. متى وكيف تكتب النتائج في السجلات والذاكرة.

هذه الأمور تحدد هل المعالج:

يعمل In-Order أو Out-of-Order

Speculation

عملية تنبؤ بنتيجة تعليمة ما لتجاوز اعتماديتها والسماح بتنفيذ تعليمات أخرى مبكراً
الهدف: زيادة التوازي وتقليل فترات الانتظار في pipeline

أنواع Speculation:

1. Speculating about the outcome of a branch (Branch Prediction)

المعالج يحاول تخمين نتيجة تعليمة الشرط (if, loop) لبدء تنفيذ التعليمات التالية قبل معرفة النتيجة الحقيقية

2. Speculating that a store before a load doesn't refer to same address (Memory Dependence Prediction)

تخمين أن عملية التخزين (store) لا تشترك بنفس العنوان مع عملية التحميل (load) التالية
 يسمح بتنفيذ load قبل store

Speculation Difficulty?

1. التخمين قد يكون خاطئاً

يجب أن يعيد المعالج التراجع عن التنفيذ الخاطئ (rollback)

2. التعليمات التي نفذت بتخمين قد تسبب أخطاء لا يجب أن تحدث إذا كان التخمين خاطئاً

مثال: Static Multiple Issue مع MIPS

هنا نتحدث عن معالج MIPS يستطيع إصدار تعليمتين في كل دورة.
 هذا يسمى:

Two-issue MIPS processor: حيث يمكن أن تكون إحدى التعليمتين:

عملية حسابية (ALU) أو فرع (Branch)

والتعليمة الثانية: Load أو Store

يعني دائماً عندنا زوج من تعليمات: حساب/فرع تحميل/تخزين

لماذا هذا التقسيم؟

لتسهيل فك الشفرة (Decoding) ولضمان قدرة المعالج على تنفيذ التعليمات في وقت واحد بدون تعارض.

ولكن لكي يصدر المعالج تعليمتين معاً:

يجب أن يكون طول التعليمات = 64 بت (لأن كل تعليمة 32 بت $\times 2$)

وماذا إذا لم تكن إحدى التعليمتين مناسبة؟

يتم وضع تعليمة وهمية:

nop (No Operation) حتى يحافظ المعالج على شكل الزوج

المعالجات ثنائية الإصدار الثابتة (Static Two-Issue Processors):

مبدأ عمله: في كل دورة ساعة: تصدر تعليمتان معاً

1. تعليمة حسابية/شرطية (ALU/branch)
2. تعليمة نقل بيانات (load/store)
3. تعليمة ALU أو branch $IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$
4. تعليمة تحميل أو تخزين $IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$

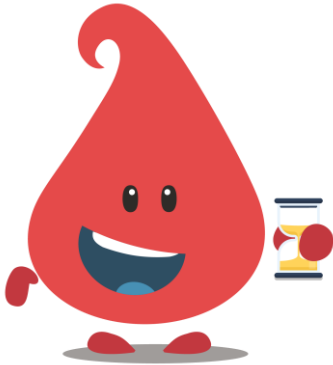
التنفيذ المتوازي:

التعليمات تسير في pipeline بشكل متواز

باختصار:

كل تعليمة تمر بخمس مراحل:

1. IF جلب التعليمة من الذاكرة
2. ID فك الترميز + قراءة السجلات
3. EX تنفيذ العملية
4. MEM الوصول للذاكرة
5. WB كتابة النتيجة للسجل



المعالج ثنائي الإصدار يدخل تعليمتين في هذه المراحل بالتوازي.

نلاحظ أنه في المعالج العادي (أحادي الإصدار) الموارد تكفي لتعليمة واحدة في كل دورة لكن في المعالج ثنائي الإصدار، نحتاج لمضاعفة بعض الموارد لتجنب Structural Hazards.

الموارد الإضافية المطلوبة:

منافذ إضافية في ملف السجلات (Extra Register File Ports)
لأنه في دورة ساعة واحدة قد نحتاج لعدة عمليات قراءة وكتابة في نفس الوقت:

1. عمليات القراءة المطلوبة:

- لتعليمة ALU: نقرأ سجلين (المصدرين)
- لتعليمة التخزين (store): نقرأ سجلين (العنوان + البيانات)
- أي 4 عمليات قراءة في نفس الدورة

2. عمليات الكتابة المطلوبة:

- لتعليمة ALU: نكتب سجلاً واحداً (النتيجة)
 - لتعليمة التحميل (load): نكتب سجلاً واحداً (البيانات المحملة)
 - أي عمليتي كتابة في نفس الدورة
- والحل: تصميم ملف سجلات بـ 4 منافذ قراءة و 2 منفذ كتابة بدلاً من 2 للقراءة و 1 للكتابة

3. دائرة جمع إضافية (Additional Adder)

والسبب أن ALU الرئيسي مشغول بتنفيذ العملية الحسابية
عمليات التحميل/التخزين تحتاج دائرة جمع لحساب العنوان الفعلي:
العنوان = قيمة السجل الأساسي + الإزاحة (Offset)

مثال:

▪ $lw \$t0, 8(\$s1) \# \text{العنوان} = \$s1 + 8$ محتوى
هذه العملية تحتاج دائرة جمع وALU مشغول بتنفيذ تعليمة أخرى في نفس الوقت

ولكن هل الأداء يضعف؟

نعم والسبب انه ستظهر مشاكل أكثر عند تنفيذ تعليمات أكثر

4. مشاكل branch (Control Hazards)

لو تنبؤ branch خطأ سنخسر تعليمتين بدل الواحدة.

5. مشكلة التحميل والاستخدام (Load-Use Hazard)

لو لدينا تعليمة load وبعدها مباشرة تعليمة تستخدم النتيجة، فيجب ان نوقف دورة كاملة لأن النتيجة لم تنتهي بعد.
في المعالج ثنائي الإصدار، المشكلة أسوأ لأن:
الدورة الثانية تحمل تعليمتين كلاهما قد يعتمدان على نتيجة الـ load

مثال:

الدورة 1: $lw \$t0, 0(\$s1) \mid add \$t2, \$t3, \$t4$
الدورة 2: $add \$t1, \$t0, \$s2 \mid sub \$t5, \$t6, \$t7$
تعليمية $add \$t1$ يجب ان تنتظر وبالتالي تتوقف كلا التعليمتين في الدورة الثانية

كيف نخفف المشاكل؟

▪ على مستوى Hardware:

- Out-of-Order Execution: المعالج يعيد ترتيب التعليمات تلقائياً.
- Bypassing/Forwarding: نقل النتائج مباشرة بدلاً من انتظار كتابتها للسجل.
- Speculation: الاعتماد على تخمين الفرع ويطابع التنفيذ قبل معرفة النتيجة.

▪ على مستوى المترجم (Compiler):

- Instruction Scheduling: إعادة ترتيب التعليمات لإبعاد التعليمات المعتمدة على بعضها.
- Loop Unrolling: فك الحلقة (تكبيرها) ليزداد التوازي (سنتكلم عنه بالتفصيل)

مقارنة في الأداء الحقيقي بين :

المعالج العادي: $CPI \approx 1$

المعالج ثنائي الإصدار المثالي ($CPI \approx 0.5$)

Don't stop when you're tired. Stop when you're done

تذكرة:

CPI (Cycles Per Instruction): هو مقياس يستخدم لتحديد عدد الدورات التي يحتاجها المعالج لتنفيذ تعليمة واحدة، كلما كان CPI أقل، كان أداء المعالج أفضل.

هذا النوع من المعالجات يمكنه تنفيذ تعليمات متعددة في نفس الوقت (توازي كامل)، مما يعني أنه يمكنه تنفيذ تعليمتين في كل دورة حيث أن كل تعليمة تحتاج إلى نصف دورة فقط.

المعالج ثنائي الإصدار الواقعي (CPI من 0.5 إلى 1) هذا النوع من المعالجات يحاول تحقيق التوازي، لكنه يواجه بعض القيود مثل الاعتماديات بين التعليمات أو قيود hardware

الأداء: يعتمد على عدة عوامل مثل:

- قابلية البرنامج للتوازي: إذا كان البرنامج مصمماً بشكل جيد للاستفادة من التوازي، فإن CPI سيكون أقرب إلى 0.5
- ذكاء المترجم: يمكن للمترجم تحسين التعليمات وإعادة ترتيبها لتحقيق توازي أفضل
- قوة hardware في إدارة الاعتماديات: إذا كان المعالج قادراً على إدارة الاعتماديات بشكل فعال (مثل استخدام تقنيات مثل التخزين المؤقت والتنبؤ بالفرع) فإنه يمكن أن يحسن من الأداء ويقلل من CPI

زمن الاستخدام (Use Latency):

عدد دورات الساعة بين تعليمة تحميل والتعليمة التي تستخدم نتيجتها دون توقف

في المعالج أحادي الإصدار:

- زمن الانتظار: دورة ساعة واحدة
- $lw \$t0, 0(\$s1)$ # دورة 1: تحميل
- (توقف) # دورة 2: انتظار
- $add \$t1, \$t0, \$s2$ # دورة 3: استخدام النتيجة

في المعالج ثنائي الإصدار:

- نفس زمن الانتظار: دورة ساعة واحدة
- لكن التأثير أكبر: التعليمتان التاليتان معاً لا تستطيعان استخدام النتيجة
- الدورة 1: $lw \$t0, 0(\$s1) \mid add \$t2, \$t3, \$t4$
- الدورة 2: $add \$t1, \$t0, \$s2 \mid sub \$t5, \$t6, \$t7$

Loop Unrolling:

تقنية يقوم فيها (compiler) بنسخ جسم الحلقة عدة مرات لإنشاء حلقة مفككة أكبر فبدلاً من تنفيذ الحلقة 100 مرة، ننفذها 25 مرة ولكن جسم الحلقة يحتوي على 4 عمليات في كل تكرار

فوائد Loop Unrolling:

- زيادة التوازي (ILP - Instruction Level Parallelism)
- عند فك الحلقة نستطيع ترتيب تعليمات من تكرارات مختلفة لتتداخل معاً
- تقليل وقت الانتظار لأن التعليمات من تكرارات مختلفة لا تعتمد على بعضها
- تقليل تكلفة التحكم في الحلقة
- تقل عدد مرات القفز (branch instructions)
- تقل التنبؤات الخاطئة للقفزات (branch mispredictions)
- إعادة تسمية السجلات (Register Renaming)

مثال:

لدينا حلقة تكرارية (loop) تقوم بمعالجة عناصر مصفوفة في معالج MIPS فيه pipeline ثنائي الإصدار (two-issue)، أي يمكنه تنفيذ تعليمتين في نفس الدورة. الحلقة الأصلية:

```

Loop: lw $t0, 0($s1) # تحميل عنصر المصفوفة
      addu $t0, $t0, $s2 # إضافة قيمة ثابتة
      sw $t0, 0($s1) # تخزين النتيجة
      addi $s1, $s1, -4 # تحديث المؤشر
      bne $s1, $zero, Loop # العودة إذا لم ننتهي
  
```

والمشكلة هي (Pipeline Stalls):

- هناك 3 data dependences تسبب توقفاً في Pipeline التعليمات تنتظر بعضها لأنها تحتاج نتائج تعليمات سابقة على رغم أن المعالج يمكنه نظرياً تنفيذ تعليمتين في نفس الوقت لكن data dependences تمنع ذلك
- كما في المثال: استخدام \$t3, \$t2, \$t1 بدلاً من \$t0 فقط
- هدفها: إزالة الاعتمادية على الأسماء (Name Dependences)
- عندما يكون لدينا نفس السجل المستخدم في عمليات مختلفة، نجبر على الانتظار بإضافة سجلات جديدة، نستطيع تنفيذ عمليات في نفس الوقت

كيف نقوم بفك الحلقات عمليا:

بدلا من :

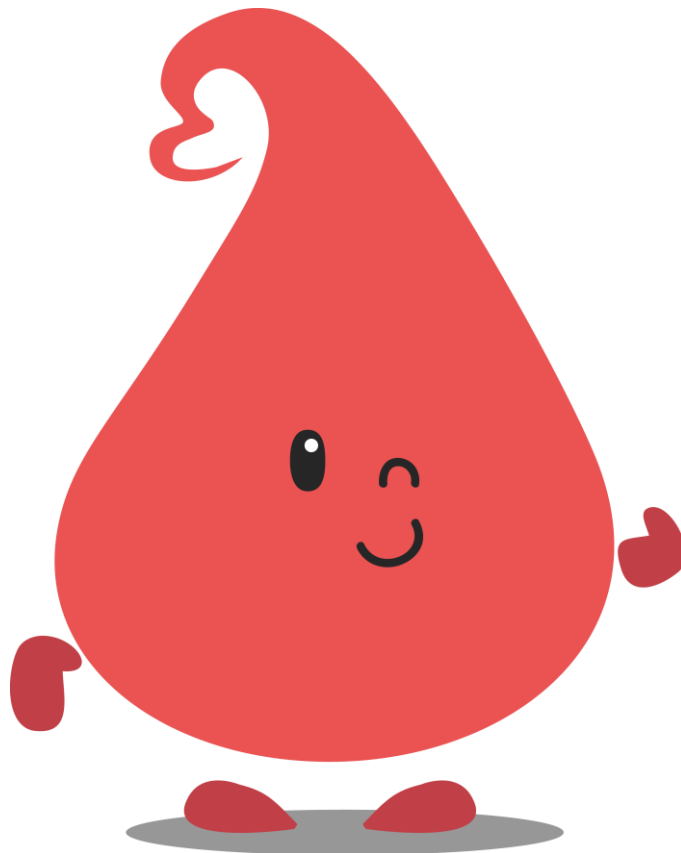
نفك الحلقة 4 مرات:

```
for(i=0; i<100; i+=4) {
  A[i] = A[i] + B;
  A[i+1] = A[i+1] + B;
  A[i+2] = A[i+2] + B;
  A[i+3] = A[i+3] + B;
}
```

```
for(i=0; i<100; i++) {
  A[i] = A[i] + B;
}
```

المشاكل التي من المحتمل تواجدها :

- زيادة حجم الكود
- استهلاك أكثر للسجلات
- قد لا يكون مناسباً إذا عدد التكرارات غير معروف في وقت الترجمة
- قد يسبب مشاكل في ذاكرة التخزين المؤقت (cache)



- The End -